

# Module 2 Lab Exercise: Tools Used in Machine Learning

## Learning Objectives

By the end of this lab, you will be able to:

- Set up and navigate Jupyter Notebook, Google Colab, and VS Code environments
- Install and import essential Python libraries for machine learning
- Create and format professional documentation using Markdown
- Initialize a GitHub repository for your ML projects
- Understand the basic workflow of data science tools

## Prerequisites

- Basic understanding of what machine learning is (Module 1)
- Access to internet for downloading tools and datasets
- A Google account (for Colab) or local Python installation

---

## Part 1: Environment Setup and Tool Overview

What are the main tools we'll use in this course?

**Jupyter Notebook/Google Colab:** Interactive computing environments where you can write code, see results immediately, and document your work with text and visualizations.

**Python Libraries:** Pre-written code packages that make machine learning tasks easier:

- **Pandas:** For working with data (like Excel, but more powerful)
- **NumPy:** For mathematical operations on arrays of numbers
- **Matplotlib:** For creating charts and graphs
- **Scikit-learn:** The main library for machine learning algorithms

**GitHub:** A platform to store, share, and collaborate on code projects

**VS Code:** A powerful text editor for writing and debugging code

Let's start by setting up our environment!

## ✓ Environment Setup Instructions

### Option 1: Google Colab (Recommended for Beginners)

1. Open [Google Colab](#).
2. Sign in and upload this `.ipynb` file.
3. Run the code cells from top to bottom. Start with the imports in Cell 3.
4. Use **Runtime > Run all** to check the complete notebook before exporting.

### Option 2: Local Jupyter Notebook

1. Install Python from [python.org](#).
2. Install the tools: `pip install jupyter pandas numpy matplotlib scikit-learn`.
3. Run `jupyter notebook` and open this notebook.

### Option 3: VS Code

1. Install [VS Code](#).
2. Install its Python and Jupyter extensions.
3. Open this `.ipynb` file and select a Python environment with the required libraries.

**How to read this notebook:** Text cells explain the work; code cells run Python. Later code cells use variables created earlier, so execution order matters.

```
# Import the tools. Aliases are short names used in later cells.
# In Colab, these libraries are normally already installed.
# If needed locally: pip install pandas numpy matplotlib scikit-learn
import pandas as pd          # Tables: rows and columns
import numpy as np           # Numerical calculations
import matplotlib.pyplot as plt # Charts
from sklearn import datasets  # Built-in practice datasets

print("All libraries imported successfully!")
print(f"Pandas version: {pd.__version__}")
print(f"NumPy version: {np.__version__}")
```

```
All libraries imported successfully!
Pandas version: 2.2.3
NumPy version: 2.1.3
```

## ✓ Part 2: Loading and Exploring Your First Dataset

We'll use the famous Iris dataset - a classic dataset for beginners. It contains measurements of iris flowers from three different species.

**Easy explanation:** One row represents one flower. A feature is a measurement, such as sepal length. The target is the flower's species. Sepals are the outer flower parts; petals are the inner flower parts.

```
from sklearn.datasets import load_iris

# Load the small dataset included with scikit-learn.
iris = load_iris()
print("Dataset loaded successfully!")
# Shape means (number of flowers, number of measurement features).
print(f"Dataset shape: {iris.data.shape}")
print(f"Features: {iris.feature_names}")
print(f"Target classes: {iris.target_names}")
```

```
Dataset loaded successfully!
Dataset shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
Target classes: ['setosa' 'versicolor' 'virginica']
```

```
# A DataFrame is a table with named columns.
df = pd.DataFrame(iris.data, columns=iris.feature_names)
# Convert numeric labels 0, 1, and 2 into readable species names.
df['species'] = iris.target_names[iris.target]

print("First 5 rows of our dataset:")
print(df.head().to_string(index=False))

# info() prints its own summary; wrapping it in print() would add None.
print("\nDataset info:")
df.info()
print(f"\nMissing values in the whole table: {df.isna().sum().sum()}")
# There are 5 table columns: 4 measurement features + 1 species label.
```

```
First 5 rows of our dataset:
```

| sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) |
|-------------------|------------------|-------------------|------------------|
| 5.1               | 3.5              | 1.4               | 0.2              |
| 4.9               | 3.0              | 1.4               | 0.2              |
| 4.7               | 3.2              | 1.3               | 0.2              |
| 4.6               | 3.1              | 1.5               | 0.2              |
| 5.0               | 3.6              | 1.4               | 0.2              |

```
Dataset info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
```

| # | Column            | Non-Null Count | Dtype   |
|---|-------------------|----------------|---------|
| 0 | sepal length (cm) | 150 non-null   | float64 |
| 1 | sepal width (cm)  | 150 non-null   | float64 |
| 2 | petal length (cm) | 150 non-null   | float64 |
| 3 | petal width (cm)  | 150 non-null   | float64 |
| 4 | species           | 150 non-null   | object  |

dtypes: float64(4), object(1)

memory usage: 6.0+ KB

Missing values in the whole table: 0

## ✓ Part 3: Creating Your First Visualization

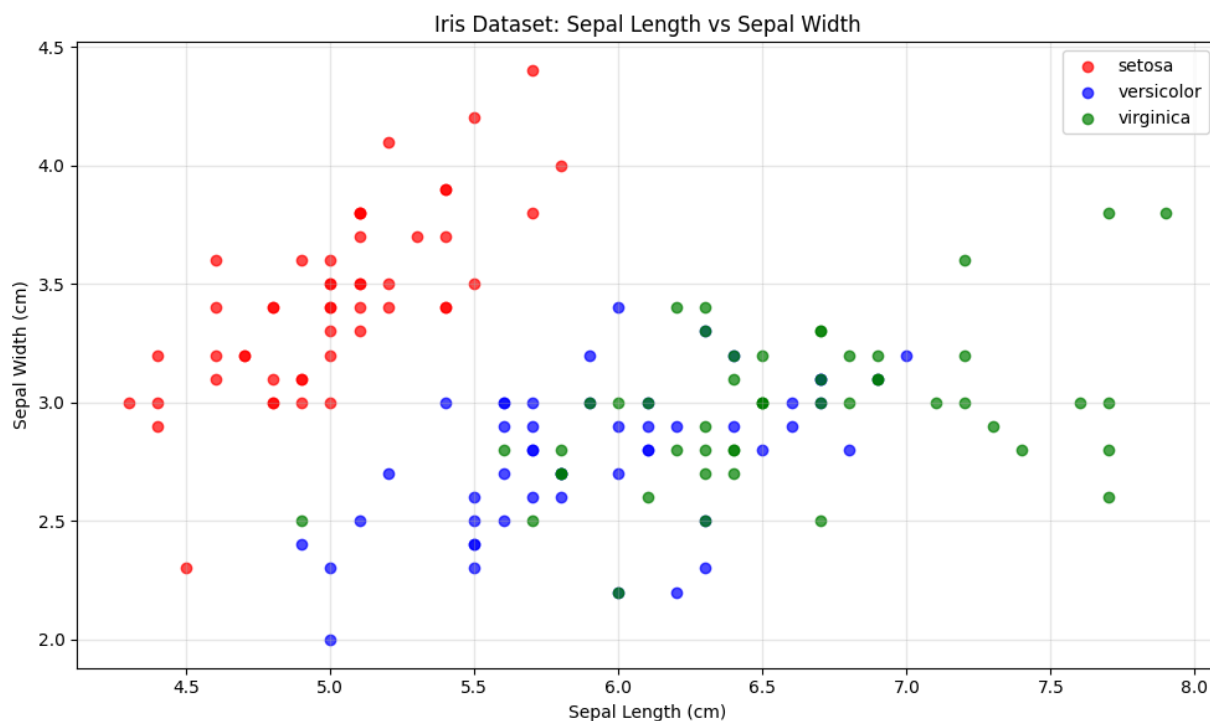
Data visualization is crucial in machine learning. Let's create a simple plot to understand our data.

**Easy explanation:** Each dot is one flower. The x-axis gives its sepal length; the y-axis gives its sepal width. Color shows species. Dots of different colors in the same area indicate that these two measurements do not separate those species perfectly.

```
plt.figure(figsize=(10, 6))
species_colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica': 'g'}

for species in df['species'].unique():
    # Keep only the rows belonging to the current species.
    species_data = df[df['species'] == species]
    plt.scatter(
        species_data['sepal length (cm)'],
        species_data['sepal width (cm)'],
        c=species_colors[species],
        label=species,
        alpha=0.7, # Transparency helps reveal overlapping points.
    )

plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Iris Dataset: Sepal Length vs Sepal Width')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("Scatter plot created successfully!")
```



## ✓ Part 4: Practice with Basic Data Operations

Let's practice some basic data analysis operations that you'll use throughout the course.

**Easy explanation:** `groupby` puts rows into groups with the same species. `mean` finds the average of each measurement within those groups. `value_counts` tells us how many flowers belong to each species.

```
print("Basic Statistics for Iris Dataset:")
print("=" * 40)

# Average each measurement separately for each species.
species_means = df.groupby('species')[iris.feature_names].mean()
print("\nMean values by species (cm):")
print(species_means.round(3).to_string())
```

```
# Count rows, rather than averaging measurement values.
species_counts = df['species'].value_counts()
print("\nSamples per species:")
print(species_counts.to_string())
print(f"Total samples: {species_counts.sum()}")
```

Basic Statistics for Iris Dataset:

=====

Mean values by species (cm):

|            | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) |
|------------|-------------------|------------------|-------------------|------------------|
| species    |                   |                  |                   |                  |
| setosa     | 5.006             | 3.428            | 1.462             |                  |
| versicolor | 5.936             | 2.770            | 4.260             |                  |
| virginica  | 6.588             | 2.974            | 5.552             |                  |

Samples per species:

|                |     |
|----------------|-----|
| species        |     |
| setosa         | 50  |
| versicolor     | 50  |
| virginica      | 50  |
| Total samples: | 150 |

## Part 5: GitHub and Documentation Best Practices

### Why GitHub for Machine Learning?

- **Version Control:** Track changes to your code and data
- **Collaboration:** Work with others on projects
- **Portfolio:** Showcase your work to potential employers
- **Backup:** Never lose your work

### Basic GitHub Workflow:

1. **Create Repository:** A folder for your project
2. **Clone/Download:** Get the project on your computer
3. **Add Files:** Put your notebooks and data
4. **Commit:** Save a snapshot of your changes
5. **Push:** Upload changes to GitHub

### For This Course:

- Create a repository named "ITAI-1371-ML-Labs"
- Upload each lab notebook as you complete it

- Include a README.md file describing your projects

**Action Item:** After this lab, create your GitHub account and repository.

## ✓ Assessment: Tool Familiarity Check

Complete the following tasks to demonstrate your understanding of the tools:

**Easy explanation:** The mean gives the average. Standard deviation measures how spread out the values are around that average. A bar chart compares counts across categories. The calculations for Task 1 and the chart for Task 2 were already supplied in the starter notebook; the comments explain how they work.

```
# Task 1: Use NumPy to summarize sepal length.
# Convert the table column into an array of numbers.
sepal_lengths = df['sepal length (cm)'].to_numpy()

mean_sepal_length = np.mean(sepal_lengths)
# ddof=0 divides by N, describing the spread of all 150 listed values.
std_sepal_length = np.std(sepal_lengths, ddof=0)

print(f"Mean sepal length: {mean_sepal_length:.2f} cm")
print(f"Standard deviation: {std_sepal_length:.2f} cm")

# Verification (don't modify)
assert isinstance(mean_sepal_length, (float, np.floating)), "Mean should
assert isinstance(std_sepal_length, (float, np.floating)), "Std should b
print("✅ Task 1 completed successfully!")
```

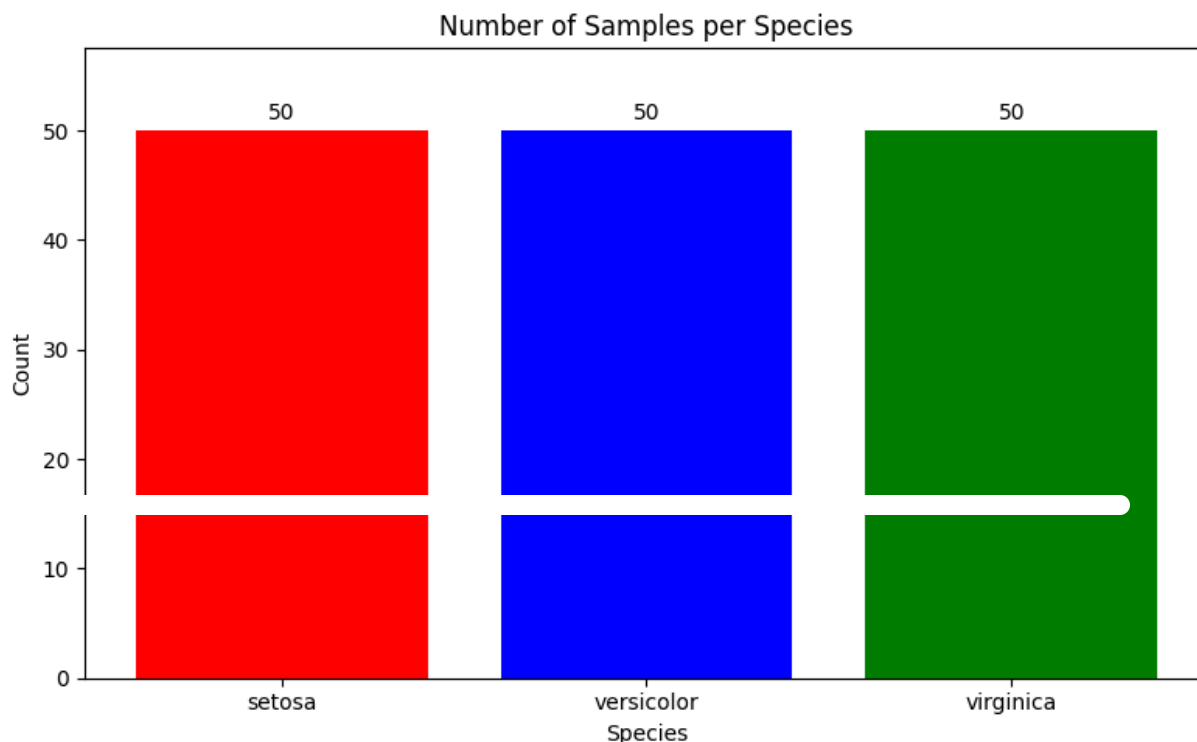
```
Mean sepal length: 5.84 cm
Standard deviation: 0.83 cm
✅ Task 1 completed successfully!
```

```
# Task 2: Compare the number of flowers in each species.
species_counts = df['species'].value_counts()
# Match each species to the same color used in the scatter plot.
bar_colors = [species_colors[name] for name in species_counts.index]

plt.figure(figsize=(8, 5))
bars = plt.bar(species_counts.index, species_counts.values, color=bar_co
plt.bar_label(bars, padding=3)
plt.ylim(0, species_counts.max() * 1.15)
plt.title('Number of Samples per Species')
plt.xlabel('Species')
plt.ylabel('Count')
plt.tight_layout()
```

```
plt.show()

print(f"Species distribution: {species_counts.to_dict()}")
print("Task 2 completed successfully!")
```



```
Species distribution: {'setosa': 50, 'versicolor': 50, 'virginica': 50}
Task 2 completed successfully!
```

## Your Analysis and Reflection

### My Observations About the Iris Dataset

#### Dataset Overview:

- Number of samples: **150 flowers**.
- Number of features: **4 measurement features**: sepal length, sepal width, petal length, and petal width, all measured in centimeters.
- Number of classes: **3 species**: setosa, versicolor, and virginica.
- Each species has **50 samples**. The DataFrame has **5 columns** because the species label is added to the four measurement columns.

## Key Findings from the Visualization:

1. Setosa flowers tend to have shorter, wider sepals than the other species. The red points form a relatively distinct cluster, with some overlap.
2. Versicolor and virginica overlap substantially on the sepal-length versus sepal-width plot. These two measurements alone do not perfectly distinguish all three species.
3. Virginica tends to have longer sepals than versicolor, but their ranges overlap. Group averages support the trend: 6.588 cm for virginica versus 5.936 cm for versicolor, compared with 5.006 cm for setosa.

## Additional Numerical Findings:

- Mean sepal length across all flowers: **5.84 cm**.
- Population standard deviation of sepal length: **0.83 cm** (NumPy `ddof=0`).
- The bar chart has three equally high bars: **50 flowers per species**.
- The table contains **no missing values**, so missing-value filling is not needed for this dataset.

## Questions for Further Investigation:

- Would a plot of petal length against petal width separate the species more clearly than the sepal measurements?
- How would a classifier using all four measurement features compare with one using only sepal length and sepal width on unseen flowers?

**Reflection:** Pandas makes flower measurements easier to inspect as a labeled table, while NumPy summarizes their average and spread. Matplotlib makes overlap between species visible, showing why averages alone cannot describe every flower. Together, these tools support checking and understanding data before building a machine learning model.

## Lab Summary and Next Steps

### What You've Accomplished:

- ✓ Set up your machine learning development environment
- ✓ Imported and used essential Python libraries
- ✓ Loaded and explored your first dataset
- ✓ Created your first data visualization
- ✓ Practiced professional documentation with Markdown
- ✓ Learned about GitHub for project management

## Preparation for Module 3:

In the next lab, you'll:

- Learn about different types of machine learning
- Build your first simple classifier
- Understand the complete ML workflow
- Work with more complex datasets

## Action Items:

1. **Create your GitHub account** and repository
2. **Upload this completed notebook** to your repository
3. **Experiment** with different visualizations using the Iris dataset
4. **Practice** Markdown formatting in a new notebook

## Resources for Continued Learning:

- [Pandas Documentation](#)
- [Matplotlib Gallery](#)
- [GitHub Guides](#)
- [Jupyter Notebook Tips](#)

**Great job completing Module 2! You're now equipped with the essential tools for machine learning.** 🎉